Reporte técnico de la práctica 05

1. Datos de identificación del estudiante y la práctica

Nombre del estudiante(s)	Alex Francisco Sigcho Serrano
Asignatura	Simulación
Ciclo	5
Unidad	U2. Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una simulación bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	05
Título de la Práctica	Implementación y análisis del Generador Congruencial Lineal en Python y Java
Nombre del Docente	José O. Guamán Q.
Fecha	29 y 30 de octubre de 2026
Horario	Jueves de 10h30 a 11h30 Viernes de 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivos de la práctica

Comprender el funcionamiento de un generador pseudoaleatorio mediante la implementación del método congruencial lineal.

1. Implementar un generador LCG en Python.
2. Implementar la misma recurrencia en Java.
3. Comparar las secuencias y analizar el efecto de la semilla y el período.

3. Materiales, reactivos, equipos y herramientas

Computador.

Python.

Java/JDK.

Sistema operativo Linux o macOS

4. Procedimiento y metodología ejecutada

Fundamento matemático

El generador calcula cada estado a partir del anterior. El módulo m restringe el estado a los enteros entre 0 y m − 1. Al dividir cada estado por m se obtiene un valor en [0, 1).

$$X_{n+1} = (a \cdot X_n + c) \bmod m \quad | \quad U_n = X_n / m$$

Se usaron a = 1 664 525, c = 1 013 904 223, m = 2^{32} y X_o = 12 345. La semilla inicial se usa para calcular X_1, que es la primera observación de la tabla.

Ejecución del experimento

1. Se implementó la recurrencia en Python y Java, separando la generación, el análisis y la ejecución. En Java se usó long porque el módulo y algunos estados exceden la capacidad de int.

2. Se generaron 10 000 estados y sus valores normalizados en Java. Después se ejecutó Python para generar otros 10 000 con los mismos parámetros y leer los CSV de Java.

3. Se compararon el índice, el estado entero y el valor normalizado de cada fila. Se mostraron las primeras 20 observaciones y se verificó la coincidencia de las 10 000.

4. Se graficaron los valores normalizados y se buscó una repetición de estado desde X_o. El período se determinó por las condiciones matemáticas del LCG.

5. Se cambió la semilla a 54 321 y se repitió el procedimiento sin modificar a, c ni m. Se compararon las dos muestras por posición.

5. Resultados

Python y Java coincidieron en las 10 000 observaciones de cada semilla. Los estadísticos descriptivos también coincidieron al nivel de precisión mostrado en la consola.

Tabla 1. Resumen de la ejecución

Semilla	Media de U	Mínimo de U	Máximo de U	Coincidencias
12345	0.4987698480	0.0000590826	0.9999259654	10 000 / 10 000
54321	0.4988696998	0.0000783911	0.9998461837	10 000 / 10 000

No se detectaron estados repetidos al revisar X_o a X_{10000} en cada ejecución. Entre las dos semillas, se registraron 0 coincidencias al comparar valores en la misma posición.

Comparación de las primeras 20 observaciones. cada valor de la tabla coincide en Java y Python. Los valores normalizados se muestran con diez decimales.

Tabla 2. Estados y valores normalizados por semilla

n	X_n (12 345)	U_n (12 345)	X_n (54 321)	U_n (54 321)
1	87628868	0.0204026857	1238253532	0.2883033669
2	71072467	0.0165478482	1708513675	0.3977943386
3	2332836374	0.5431557945	3683344750	0.8575955289
4	2726892157	0.6349040561	1863498229	0.4338794921
5	3908547000	0.9100295138	2137457360	0.4976655729
6	483019191	0.1124616691	102299631	0.0238184889
7	2129828778	0.4958894053	3033777282	0.7063563173
8	2355140353	0.5483488443	4230855161	0.9850727303
9	2560230508	0.5961001171	1814797060	0.4225403676
10	3364893915	0.7834504161	1036872339	0.2414156541
11	171172990	0.0398543174	2695818966	0.6276692650
12	3194601925	0.7438012224	1801476157	0.4194388532
13	4148119648	0.9658093676	2976955512	0.6931264680
14	316399679	0.0736675409	2448971127	0.5701955238

n	X_n (12 345)	U_n (12 345)	X_n (54 321)	U_n (54 321)
15	3004788882	0.6996069294	4038571114	0.9403031119
16	1976948425	0.4602941743	1174362305	0.2734275314
17	1702883732	0.3964835154	849107756	0.1976983054
18	4121112547	0.9595212869	33496219	0.0077989462
19	1744294886	0.4061253010	3337365822	0.7770410324
20	4092090893	0.9527641565	1978253189	0.4605979633

Efecto del cambio de semilla

La figura muestra cómo cambia el orden de los valores al cambiar el estado inicial. Ambas series permanecen en el intervalo [0, 1).

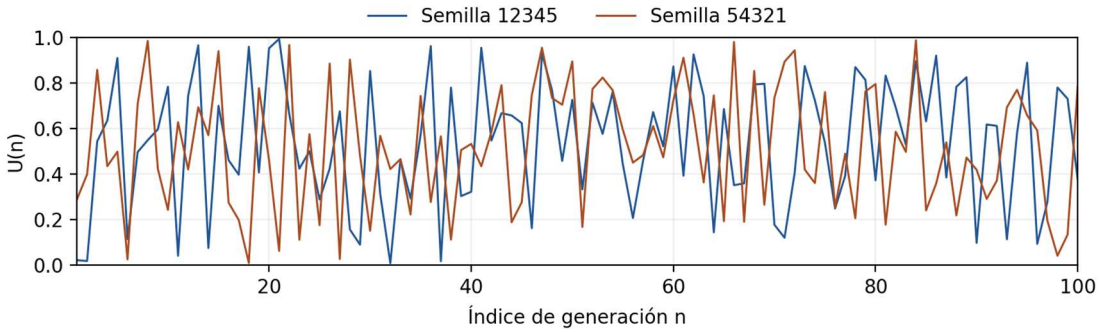


Figura 1. Primeros 100 valores por semilla. Gráfica regenerada a partir de los parámetros de la práctica y contrastada con los datos de consola.

Análisis del período

Según el teorema de Hull y Dobell [2], para $m = 2^k$ con $k \geq 2$ el LCG mixto tiene período completo si c es impar y $a - 1$ es divisible por 4. El incremento 1 013 904 223 es impar y $a - 1 = 1\,664\,524 = 4 \times 416\,131$. Por tanto, el período teórico es $2^{32} = 4\,294\,967\,296$ estados para ambas semillas.

La búsqueda de repeticiones abarcó los primeros 10 000 pasos. El período completo se obtiene del teorema. Ambas semillas pertenecen al mismo ciclo y cambian el punto desde el que se recorre. El resultado de 0 coincidencias por posición no permite inferir independencia estadística.

6. Preguntas de control

1. ¿Qué significa pseudoaleatorio?

Que los valores tienen apariencia aleatoria, aunque se calculan con una regla determinista y pueden reproducirse.

2. ¿Qué función cumple la semilla?

Define el estado inicial X_o. Con la misma semilla y los mismos parámetros se recupera la misma secuencia.

3. ¿Qué significa período?

Es el número de pasos que se requieren para regresar a un estado del ciclo y volver a producir la misma sucesión.

4. ¿Qué ocurre si se utilizan parámetros inadecuados?

Pueden aparecer ciclos cortos, estados que nunca se visitan o patrones que afectan la utilidad de los valores en una simulación.

5. ¿Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes?

Permite contrastar los resultados y detectar errores de implementación, como usar un tipo entero insuficiente o dividir sin convertir a punto flotante.

7. Conclusiones

Las implementaciones produjeron los mismos estados y valores normalizados en las 20 000 filas comparadas, 10 000 por semilla. Esto confirma la consistencia del algoritmo para las configuraciones evaluadas.

El cambio de semilla alteró la secuencia desde la primera observación y mantuvo el período teórico. Las medias cercanas a 0,5 describen las muestras y se necesitarían pruebas adicionales para evaluar uniformidad e independencia.

8. Recomendaciones

Registrar los parámetros y la semilla en cada experimento para reproducirlo. Conservar los CSV y revisar los primeros estados al cambiar de lenguaje o de tipo numérico. Para evaluar la calidad de la secuencia en una simulación, complementar las gráficas con pruebas de uniformidad e independencia.

9. Reflexión crítica

La práctica me permitió relacionar el estado entero del generador con el valor normalizado que se usa en simulación. Al comparar Java y Python comprendí por qué deben coincidir la semilla, la posición de inicio y los parámetros. También pude distinguir entre lo observado en una muestra y lo que se deduce de las condiciones matemáticas del generador.

10. Bibliografía y referencias

[1] J. O. Guamán Q., «Implementación y análisis del Generador Congruencial Lineal en Python y Java», Guía de Actividades Práctico-Experimentales Nro. 004, práctica 05, Universidad Nacional de Loja, oct. 2026.

[2] T. E. Hull y A. R. Dobell, «Random Number Generators», SIAM Review, vol. 4, n.º 3, pp. 230–254, jul. 1962, doi: 10.1137/1004061.

11. Anexos